

COMPUTER SCIENCE — Assignment 1: Von Neumann Architecture & Signed Binary (1's/2's Complement)

Quick Recap (for reference while solving)

Von Neumann Architecture

A computer built on the **Von Neumann model** has these core components sharing a single memory for both instructions and data:

Component	Role
CPU — Control Unit (CU)	Fetches instructions, decodes them, directs other components
CPU — Arithmetic Logic Unit (ALU)	Performs arithmetic (+, −, ×) and logic (AND, OR, NOT) operations
Registers	Tiny, ultra-fast storage inside the CPU: e.g. PC (Program Counter — address of next instruction), MAR (Memory Address Register), MDR (Memory Data Register), IR (Instruction Register), Accumulator
Memory (RAM)	Stores <i>both</i> program instructions and data — the defining feature of Von Neumann architecture
Buses	Address bus (carries the memory location to access), Data bus (carries the actual data), Control bus (carries signals like read/write)

Fetch-Decode-Execute cycle:

Fetch: PC holds the address of the next instruction → address copied to MAR → instruction fetched from memory into MDR → copied into IR → PC incremented.

Decode: CU interprets what the instruction in IR means.

Execute: CU/ALU carries out the instruction (e.g. add two registers, store a value).

Von Neumann bottleneck: since instructions and data share the *same* memory and the *same* bus, the CPU cannot fetch an instruction and read/write data at exactly the same time — this shared pathway limits overall speed.

1's Complement

To get the 1's complement of a binary number, **flip every bit** (0→1, 1→0).

Used to represent negative numbers, but has **two representations of zero** (+0 = 00000000, −0 = 11111111) — a flaw.

2's Complement (the one actually used in real computers)

To get the 2's complement: **flip every bit, then add 1**.

Only **one representation of zero**.

For an n-bit number, range is: $-2^{(n-1)}$ to $+2^{(n-1)} - 1$

e.g., 8 bits $\rightarrow$ -128 to +127

The leftmost (most significant) bit is the **sign bit**: 0 = positive, 1 = negative.

To convert a 2's complement number back to decimal: if sign bit is 1, the value is negative — flip bits, add 1, convert to decimal, apply a negative sign.

Part A — Von Neumann Architecture (Conceptual)

A1. Draw (describe in words if you can't draw) the block diagram of the Von Neumann architecture, labeling: CPU, Control Unit, ALU, Registers, Memory, and the three buses.

A2. Explain, step-by-step, what happens during the **fetch** stage of the fetch-decode-execute cycle. Name every register involved.

A3. What is the "Von Neumann bottleneck"? Why does storing instructions and data in the same memory create this limitation?

A4. What is the difference between the **Program Counter (PC)** and the **Instruction Register (IR)**? Why does a computer need both?

A5. True or False (justify each answer in one sentence): a) In Von Neumann architecture, instructions and data are stored in physically separate memory units. b) The ALU is responsible for decoding instructions. c) The address bus carries the actual data being read or written. d) The Control Unit directs the flow of data between the CPU, memory, and I/O devices.

A6. Research/short-answer: Name one alternative to Von Neumann architecture (e.g. Harvard architecture) and state **one** key difference.

Part B — 1's Complement & 2's Complement (Computational)

B1. Find the 1's complement of the following 8-bit binary numbers: a) 01001101 b) 00000000 c) 11111111

B2. Find the 2's complement of the following 8-bit binary numbers: a) 00101011 b) 01111111 c) 10000000

B3. Convert the following decimal numbers into their 8-bit 2's complement binary representation: a) +25 b) -25 c) -1 d) -128

B4. Convert the following 8-bit 2's complement binary numbers into decimal: a) 00010110 b) 11101010 c) 10000001 d) 11111111

B5. Perform the following 2's complement binary additions (8-bit), and state whether **overflow** occurs (overflow happens when adding two numbers of the *same sign* produces a result with the *opposite* sign): a) 00000101 + 00000011 b) 01111111 + 00000001 c) 11111111 + 11111111 d) 10000000 + 11111111

B6. Explain in your own words why 2's complement is preferred over 1's complement in real hardware. Mention the "two zeros" problem and how addition/subtraction circuitry benefits.

Part C — Mixed / Applied

C1. A computer has an 8-bit register storing the value 11010110 in 2's complement form. What decimal value does it represent? If you add 00000010 to it, what is the new binary value and its decimal equivalent? Did overflow occur?

C2. Explain how subtraction ($A - B$) can be performed using only an adder circuit, by using 2's complement. (Hint: $A - B = A + (-B)$, and $-B$ is obtained via 2's complement of B .)

C3. If a system used **4-bit** 2's complement numbers instead of 8-bit, what would the representable range be? List all 16 possible bit patterns and their decimal values in a table.